🔑 Lesson 1 – What is Node.js?

Node.js allows you to run **JavaScript outside the browser**.

Before Node:

JavaScript worked only in browser.

Example:

Browser → JavaScript → UI

After Node:

Server → JavaScript → Backend

Example backend tasks:

- Handle API requests
- Connect database
- Authentication
- File uploads

Node uses the **V8 JavaScript Engine** from **Google**.

⚡ Example Node Code

Create file:

app.js

Code:

```
console.log("Hello from Node.js");
```

Run in terminal:

```
node app.js
```

Output:

```
Hello from Node.js
```

□ Interview Question

Q: Why Node.js is fast?

Answer:

Because it uses

- **Non-blocking I/O**
- **Event-driven architecture**
- **Single-threaded event loop**

> Your Task

1 Install **Node.js**
2 Check version in terminal:

node -v

and

npm -v

Next Lesson (Lesson 2)

I will teach you:

Node.js Architecture

Event Loop (Very Important)
Blocking vs Non-Blocking code

💬 Before moving to the next lesson tell me:

1 **Did you install Node.js?**
2 Do you use **VS Code** or another editor?

Then we start **real backend coding** 🔥

Great 👍 Let's move to **Lesson 2 of Node.js**.
This lesson is **very important for interviews**.

🚀 **Lesson 2 – Node.js Architecture & Event Loop**

1 Node.js Architecture

Node.js uses a **Single-Threaded Event-Driven Architecture**.

This means:

- Only **one main thread**
- Uses **events** to handle multiple users

Example:

User1 → Request

User2 → Request

User3 → Request

|

v

Node.js

Node handles all requests **without creating multiple threads**.

⚡ 2 What is Event Loop?

The **Event Loop** is the **heart of Node.js**.

It continuously checks:

Is there any request?

Is there any callback?

Is there any task to execute?

If yes → execute it.

Simple flow:

Request → Event Queue → Event Loop → Execute

So Node can handle **thousands of requests simultaneously**.

□ Example

Imagine a restaurant:

Chef = **Node.js**

Customers place orders.

Chef does:

1 Take order
2 Send cooking request
3 Move to next customer

Chef **doesn't wait** for food to cook.

This is called **Non-Blocking**.

● Blocking vs Non-Blocking Code

Blocking Code

Program waits for task to finish.

Example:

```
const fs = require("fs");

const data = fs.readFileSync("file.txt", "utf8");

console.log(data);

console.log("Program finished");
```

Here:

Read file → wait → then run next line

Non-Blocking Code (Preferred in Node)

Example:

```
const fs = require("fs");

fs.readFile("file.txt", "utf8", (err, data) => {

  console.log(data);

});

console.log("Program finished");
```

Output order:

Program finished

(file data later)

Node **does not wait**.

📦 What is a Module?

In **Node.js**, every file is a **module**.

Example:

math.js

app.js

You can export and import code between files.

Example:

math.js

```
function add(a, b) {
  return a + b;
}
module.exports = add;
```

---

app.js

```
const add = require("./math");
console.log(add(5, 3));
```

Run:

```
node app.js
```

Output:

```
8
```

🧠 Interview Questions

❓ Why Node.js is fast?

Answer:

Because it uses

- **Event Loop**
- **Non-blocking I/O**

- **Single thread**

📝 **Practice Task**

Create two files:

math.js

```
function multiply(a, b) {
  return a * b;
}
module.exports = multiply;
```

app.js

```
const multiply = require("./math")
console.log(multiply(4, 5));
```

Run:

```
node app.js
```

⏭ **Next Lesson (Lesson 3)**

We will learn **Core Modules in Node.js**:

- **FS module** (file system)
- **Path module**
- **OS module**

These are used in **real backend development**.

💬 When you finish the practice, say **"next lesson"** and we will continue.

Good 👍 Let's continue **Lesson 3 of Node.js**.

Today we will learn **Core Modules**. These are **built-in modules** in Node.js that help you work with files, system info, and file paths.

🚀 **Lesson 3 – Core Modules in Node.js**

1 What are Core Modules?

Core modules are modules already included in **Node.js**.

You **do not need to install them** with npm.

Example core modules:

- **FS module** → file system
- **Path module** → file paths
- **OS module** → system information

🗁 **2 FS Module (File System)**

The **FS module** is used to **create, read, update, and delete files**.

Import it like this:

```
const fs = require("fs");
```

🕮 **Read a File**

Example:

```
const fs = require("fs");
fs.readFile("test.txt", "utf8", (err, data) => {
  if (err) {
    console.log(err);
  } else {
    console.log(data);
  }
});
```

Explanation:

readFile(fileName, encoding, callback)

✍ **Create a File**

Example:

```
const fs = require("fs");
fs.writeFile("test.txt", "Hello Node.js", (err) => {
  if (err) {
    console.log(err);
  } else {
    console.log("File created");
  }
});
```

❌ Delete a File

Example:

```
const fs = require("fs");
fs.unlink("test.txt", (err) => {
  if (err) {
    console.log(err);
  } else {
    console.log("File deleted");
  }
});
```

🗀 Path Module

The **Path module** helps manage **file paths**.

Import it:

```
const path = require("path");
```

Example:

```
const path = require("path");


console.log(path.basename(__filename));
```

Output example:

app.js

Useful methods:

path.basename()

path.dirname()

path.extname()

Example:

console.log(path.extname("app.js"));

Output:

.js

💻 4 OS Module

The **OS module** gives **system information**.

Import it:

const os = require("os");

Example:

```
const os = require("os");

console.log(os.platform());
console.log(os.arch());
console.log(os.cpus());
```

Example output:

win32

x64

CPU information

□ Interview Questions

❓ What are core modules in Node.js?

Answer:

Core modules are **built-in modules provided by Node.js that do not require installation**.

Examples:

- fs
- path
- os

📝 **Practice Task**

Create file **app.js**

Try this:

```
const os = require("os");

console.log("Platform:", os.platform());

console.log("CPU:", os.arch());
```

Run:

```
node app.js
```

⏭ **Next Lesson (Very Important)**

Next we will learn:

🔥 **NPM (Node Package Manager)**
🔥 Installing packages
🔥 Creating a Node project
🔥 package.json

These are used in **every real backend project**.

When you're ready, type **“next lesson”** and we’ll continue 🚀

Great 👍 Let’s continue.

Good 👍 Let's continue **Lesson 4 of Node.js**.

Today we will learn **NPM and Node Project Setup**, which is used in **every real backend project**.

🚀 Lesson 4 – NPM & Node Project Setup

1 What is NPM?

npm is the **package manager for Node.js**.

It is used to:

- Install libraries
- Manage dependencies
- Run project scripts

Example libraries you will install later:

- **Express.js**
- **Mongoose**
- **bcrypt**

📦 2 Creating a Node Project

Create a new folder:

node-project

Open terminal inside the folder and run:

npm init

It will ask questions:

package name

version

description

entry point

Press **Enter** for default values.

📄 3 package.json

After running npm init, Node creates a file:

package.json

Example:

```
{
  "name": "node-project",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  }
}
```

This file stores:

- project information
- installed packages
- scripts

> 4 Running Scripts

Inside **package.json**:

```
"scripts": {
  "start": "node app.js"
}
```

Now instead of running:

node app.js

You can run:

npm start

📥 5 Installing Packages

Example installing **Express.js**:

npm install express

Shortcut:

npm i express

After installing, two things appear:

node_modules

package-lock.json

🗀 Node Project Structure

Typical project:

```
node-project
│
├── node_modules
├── package.json
├── package-lock.json
└── app.js
```

□ Interview Question

❓ What is package.json?

Answer:

package.json is a file that **stores project information, dependencies, and scripts in a Node.js project**.

📝 Practice Task

Do this on your computer:

1 Create folder **node-app**

2 Run:

npm init -y

3 Create **app.js**

console.log("My first Node project");

4 Run:

node app.js

⏭ **Next Lesson (Important)**

Next we will learn:

🔥 **Creating a Server using HTTP Module**
🔥 Request & Response
🔥 First Backend API

This is where **real backend development starts**.

When you are ready, type **"next lesson"** and we will build your **first Node.js server** 🚀

Good question 👍 This line is inside **package.json**.

"scripts": {

  "start": "node app.js"

}

Let me explain simply.

📜 **What is "scripts" in npm?**

In a **Node.js** project, the scripts section lets you create **custom commands**.

Instead of typing long commands in the terminal, you can run them using **npm**.

💻 **Example Without Scripts**

Normally to run your file you write:

node app.js

💻 **Example With Scripts**

If your package.json has this:

```
{
  "scripts": {
    "start": "node app.js"
  }
}
```

Now you can run:

```
npm start
```

This will automatically run:

```
node app.js
```

📄 Real Example of package.json

```
{
  "name": "node-project",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  }
}
```

Meaning:

Part	Meaning
name	project name
version	project version
main	main file
scripts	commands you can run

🚀 Why Scripts Are Useful

In real projects you will see:

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

Then you can run:

```
npm start
```

or

```
npm run dev
```

🧠 Interview Question

❓ What is the scripts section in package.json?

Answer:
The scripts section is used to **define custom commands that can be run using npm**.

⏭ Next Lesson

In the next lesson we will build your **first backend server using the HTTP module in Node.js**.

You will learn:

- Creating a server
- Request & Response
- API basics

Just type **"next lesson"** and we start building your **first Node server** 🚀.

Great 👍 Now we start **real backend development**.

This is **Lesson 5 – Creating Your First Server** using **Node.js**.

🚀 Lesson 5 – HTTP Module (Create Server)

1⃣What is HTTP Module?

The **HTTP module** is a **built-in module** in **Node.js** used to create **web servers**.

You don't need to install it.

It helps you:

- Create backend servers
- Handle requests
- Send responses

🌐 How a Server Works

When a user opens a website:

Browser → Request → Server → Response → Browser

Example:

User opens: http://localhost:3000

Server sends response like:

Hello from server

💻 Creating Your First Server

Create a file:

app.js

Write this code:

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.write("Hello from Node.js server");
  res.end();
});

server.listen(3000, () => {
```

```
  console.log("Server running on port 3000");
});
```

▶ Run the Server

In terminal run:

```
node app.js
```

Output:

```
Server running on port 3000
```

Now open browser:

http://localhost:3000

You will see:

```
Hello from Node.js server
```

🎉 Your **first backend server** is running!

🔍 Understanding the Code

Import HTTP module

```
const http = require("http");
```

Create server

```
http.createServer()
```

This creates a **server**.

Request & Response

(req, res)

Parameter	Meaning
req	request from user
res	response sent to user

Start server

server.listen(3000)

This means server runs on **port 3000**.

□ Interview Question

❓ **What is the HTTP module in Node.js?**

Answer:

The **HTTP module** in **Node.js** is a built-in module used to **create web servers and handle HTTP requests and responses**.

📝 **Practice Task**

Create this server:

```
const http = require("http");


const server = http.createServer((req, res) => {
  res.write("My first backend API");
  res.end();
});


server.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

Open:

http://localhost:5000

⏭ **Next Lesson (Very Important)**

Next we will learn:

🔥 **Routing in Node.js**
🔥 Different URLs
🔥 Building real APIs

Example:

/home

/about

/contact

Each route will return different data.

Just type **“next lesson”** and we will build your **first API routes** 🚀.

You are now in **Lesson 6 of Node.js**.

Today we will learn **Routing**, which means **different URLs returning different responses**.

🚀 **Lesson 6 – Routing in Node.js**

1 What is Routing?

Routing means handling **different URLs (routes)** in a server.

Example URLs:

/

/about

/contact

Each route should return **different data**.

Example:

/ → Home Page

/about → About Page

/contact → Contact Page

💻 **Basic Routing Example**

Create **app.js**

```
const http = require("http");
const server = http.createServer((req, res) => {
  if (req.url === "/") {
    res.write("Welcome to Home Page");
    res.end();
  }
  else if (req.url === "/about") {
    res.write("Welcome to About Page");
    res.end();
  }
  else if (req.url === "/contact") {
    res.write("Welcome to Contact Page");
    res.end();
  }
  else {
    res.write("404 Page Not Found");
    res.end();
  }
});
server.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

▶ Run the Server

Run:

```
node app.js
```

Open browser:

http://localhost:3000/

Output:

Welcome to Home Page

Try other routes:

http://localhost:3000/about

Welcome to About Page

http://localhost:3000/contact

Welcome to Contact Page

□ How This Works

Node checks the URL using:

req.url

Example values:

/

/about

/contact

Then it sends the correct response.

⚠ **Problem With This Method**

If your website has **many routes**, this method becomes messy.

Example:

/users

/products

/orders

/cart

/login

/register

That is why developers use **Express.js**, which makes routing **very easy**.

We will start **Express soon**.

📝 Practice Task

Create these routes:

/

/api

/api/users

Expected output:

/ → Home Page

/api → API Home

/api/users → Users API

📊 Course Progress

35% Complete

Completed:

✓ Node introduction
✓ Event loop
✓ Core modules
✓ NPM
✓ HTTP server
✓ Routing

⏭ Next Lesson (Important)

Next we will learn:

🔥 **Sending JSON Data (First API)**
🔥 How APIs send data to **React**
🔥 API response format

You will build your **first real backend API**.

Type **“next lesson”** and we continue 🚀

No problem 👍 I will explain **Lesson 7 again in a simpler way**.

We are learning how **Node.js** sends **data from backend to frontend**.

---

1 What is JSON?

JSON (JavaScript Object Notation) is a **data format** used to send data between server and frontend like **React**.

Example JSON:

```
{
  "name": "Kartik",
  "age": 21
}
```

or

```
[
  { "id": 1, "name": "Kartik" },
  { "id": 2, "name": "Rahul" }
]
```

APIs usually send **JSON data**.

---

2 Example API URL

Example route:

/api/users

Full URL:

http://localhost:3000/api/users

When someone opens this URL, the server sends **JSON data**.

---

3 Code Example

Create file **api.js**

```
const http = require("http");
const server = http.createServer((req, res) =>
  if (req.url === "/api/users") {
    const users = [
      { id: 1, name: "Kartik" },
      { id: 2, name: "Rahul" },
      { id: 3, name: "Aman" }
    ];

    res.writeHead(200, { "Content-Type": "application/json" });
    res.write(JSON.stringify(users));
    res.end();
  }
});
server.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

---

4 Step-by-Step Explanation

Import HTTP module

```
const http = require("http");
```

This loads the built-in **server module** in **Node.js**.

Create server

```
http.createServer((req, res) => {
```

Parameter	Meaning
req	request from browser
res	response from server

Check URL

if (req.url === "/api/users")

If user opens:

http://localhost:3000/api/users

Then this block runs.

Data to send

```
const users = [
 { id: 1, name: "Kartik" },
 { id: 2, name: "Rahul" },
 { id: 3, name: "Aman" }
];
```

This is **JavaScript data**.

Convert to JSON

JSON.stringify(users)

This converts the object into **JSON format**.

Example result:

```
[
 {"id":1,"name":"Kartik"},
 {"id":2,"name":"Rahul"},
 {"id":3,"name":"Aman"}
]
```

Send response type

res.writeHead(200, { "Content-Type": "application/json" });

Meaning:

Part	Meaning
200	success status
application/json	data type

Send data

res.write(JSON.stringify(users));

This sends the data to the browser.

End response

res.end();

This finishes the response.

---

5 Run the Program

Run in terminal:

node api.js

Open browser:

http://localhost:3000/api/users

You will see JSON data.

□ Why This is Important

Every backend API works like this.

Example APIs used by **React** apps:

GET /users

GET /products

GET /orders

📊 **Your Progress**

40% Complete

If you want, I can also explain 🔥 **the API code visually step-by-step (diagram style)** which makes Node.js **much easier to understand**.

Great question 👍 This line is **very important in APIs** when using **Node.js**.

Let's break this line step-by-step.

res.writeHead(200, { "Content-Type": "application/json" });

---

1 What is res?

In the server function:

http.createServer((req, res) => {

Variable	Meaning
req	request coming from browser
res	response sent back to browser

So **res controls what the server sends to the user**.

---

2 What is writeHead()?

writeHead() is used to **send HTTP response headers**.

Headers tell the browser **what type of response is coming**.

Structure:

res.writeHead(statusCode, headers);

---

3 First Parameter → 200

res.writeHead(200)

200 is the **HTTP status code**.

Meaning: **Request successful**

Common status codes:

Code	Meaning
200	Success
404	Page not found
500	Server error

Example:

res.writeHead(404);

Means **page not found**.

---

4 Second Parameter → Headers

{ "Content-Type": "application/json" }

This tells the browser **what type of data the server is sending**.

Structure:

"Content-Type": "data type"

Common types:

Content-Type	Meaning
text/plain	normal text
text/html	HTML page
application/json	JSON data

---

5 Example

Sending plain text

res.writeHead(200, { "Content-Type": "text/plain" });

res.write("Hello World");

Browser shows:

Hello World

Sending HTML

```
res.writeHead(200, { "Content-Type": "text/html" });
res.write("<h1>Hello</h1>");
```

Browser shows a **heading**.

Sending JSON (API)

```
res.writeHead(200, { "Content-Type": "application/json" });
res.write(JSON.stringify({ name: "Kartik" }));
```

Output:

```
{"name":"Kartik"}
```

---

6 Simple Way to Remember

```
res.writeHead(
  200,            → request success
  Content-Type    → type of data
)
```

Example:

Server → Browser

Status: 200

Data: JSON

📊 **Your Course Progress**

40% Complete

⏭ **Next Lesson (Very Important)**

Next we will learn **Events and EventEmitter** in **Node.js**.

This explains **how Node handles asynchronous events internally**.

Just type **"next lesson"** and we continue 🚀

Great 👍 Let's continue.

You are now in **Lesson 8 of Node.js**.

Today we learn **Events and EventEmitter**, which is a **core concept of Node.js architecture**.

🚀 **Lesson 8 – Events & EventEmitter**

1 What is an Event?

An **event** is something that **happens in the application**.

Examples:

User clicks button

File is uploaded

Server receives request

When an event happens → Node can **trigger a function**.

---

2 Event-Driven Architecture

Node.js works using an **event-driven system**.

Flow:

Event happens → Event detected → Function runs

Example:

User login → login event → run login function

---

3 EventEmitter Module

Node provides **EventEmitter** to create and handle events.

Import it like this:

```
const EventEmitter = require("events");
```

---

4 Create Event Object

```
const EventEmitter = require("events");


const eventEmitter = new EventEmitter();
```

Now eventEmitter can **create and listen to events**.

---

5 Listening to an Event

Use .on() method.

```
eventEmitter.on("greet", () => {
  console.log("Hello Kartik!");
});
```

Meaning:

When "greet" event happens → run this function

---

6 Emitting an Event

Use .emit() to trigger the event.

```
eventEmitter.emit("greet");
```

---

7 Full Example

Create file:

events.js

Code:

```
const EventEmitter = require("events");
const eventEmitter = new EventEmitter();
eventEmitter.on("greet", () => {
  console.log("Hello from Node Event!");
});
```

```
eventEmitter.emit("greet");
```

Run:

```
node events.js
```

Output:

```
Hello from Node Event!
```

---

8 Event with Data

Events can also send data.

Example:

```
const EventEmitter = require("events");
const eventEmitter = new EventEmitter();
eventEmitter.on("userLogin", (username) => {
  console.log(username + " logged in");
});
eventEmitter.emit("userLogin", "Kartik");
```

Output:

```
Kartik logged in
```

□ Interview Question

❓ **What is EventEmitter in Node.js?**

Answer:

EventEmitter is a class in Node.js that allows objects to emit events and listen to them.

📝 **Practice Task**

Create event:

userRegister

When triggered it should print:

User registered successfully

📊 **Course Progress**

45% Complete

Completed:

✓ Node basics
✓ Event loop
✓ Core modules
✓ NPM
✓ HTTP server
✓ Routing
✓ JSON API
✓ Events

⏭ **Next Lesson (Important)**

Next we will learn:

🔥 **Streams in Node.js**
🔥 Handling large files
🔥 Performance optimization

Streams are used in **real backend systems**.

Type **"next lesson"** and we continue 🚀

Great 👍 Let's continue.

You are now in **Lesson 9 of Node.js**.

Today we will learn **Streams**, which are used to **handle large data efficiently**.

🚀 **Lesson 9 – Streams in Node.js**

1 What is a Stream?

A **stream** is a way to **process data piece-by-piece instead of loading everything at once**.

Example:

Imagine a **1 GB file**.

Two ways to read it:

❌ Load entire file → high memory usage
✅ Read small chunks → low memory usage

Streams use the **second method**.

Flow:

File → small chunks → process → output

---

2 Why Streams Are Important

In **Node.js**, streams help with:

- Reading large files
- Video streaming
- File uploads
- Network data transfer

Example: **video streaming platforms**.

---

3 Types of Streams

There are **4 main types**:

Stream Type	Purpose
Readable	read data
Writable	write data
Duplex	read & write
Transform	modify data while streaming

---

4 Example Without Stream

```
const fs = require("fs");

const data = fs.readFileSync("bigfile.txt");

console.log(data);
```

Problem:

Loads full file into memory

Bad for **large files**.

---

5 Example Using Stream

```
const fs = require("fs");

const readStream = fs.createReadStream("bigfile.txt", "utf8")

readStream.on("data", (chunk) => {

  console.log(chunk);

});
```

Here:

File → chunks → console

So Node reads **small pieces of the file**.

---

6 Understanding the Code

Create Stream

```
const readStream = fs.createReadStream("bigfile.txt");
```

This creates a **readable stream**.

Listen for data

```
readStream.on("data", (chunk) => {
```

Each **chunk** is a small part of the file.

Example:

chunk1

chunk2

chunk3

---

7 Stream Example With Pipe

You can send data directly from one stream to another.

```
const fs = require("fs");

const readStream = fs.createReadStream("input.txt");

const writeStream = fs.createWriteStream("output.txt");

readStream.pipe(writeStream);
```

Flow:

input.txt → readStream → writeStream → output.txt

🎯 Interview Question

❓ Why are streams used in Node.js?

Answer:

Streams allow **Node.js** to **process large data efficiently by reading it in chunks instead of loading the entire file into memory**.

📝 Practice Task

Create file:

stream.js

Write code that:

- Reads input.txt
- Prints chunks of data in the console.

📊 Course Progress

50% Complete

Completed topics:

✓ Node introduction
✓ Event loop
✓ Core modules
✓ NPM

✓ HTTP server
✓ Routing
✓ JSON APIs
✓ EventEmitter
✓ Streams

⏭ **Next Lesson (Important)**

Next we will learn:

🔥 **Buffers in Node.js**
🔥 Binary data handling
🔥 How Node processes files internally

Buffers are another **core concept of Node.js**.

Type **"next lesson"** and we continue 🚀 Good question 👍 Let's understand the **whole code step-by-step** in **Node.js**.

Here is the code:

```
const fs = require("fs");


const readStream = fs.createReadStream("input.txt");

const writeStream = fs.createWriteStream("output.txt");


readStream.pipe(writeStream);
```

1️⃣ Import File System Module

```
const fs = require("fs");
```

fs means **File System module**.

It is a built-in module in **Node.js** used to:

- Read files
- Write files

- Create files
- Delete files

2️⃣ Create Read Stream

```
const readStream = fs.createReadStream("input.txt");
```

This creates a **Readable Stream**.

Meaning:

Node will read input.txt little by little (chunks)

Example flow:

```
input.txt
  ↓
chunk1
chunk2
chunk3
```

Instead of loading the **entire file in memory**, Node reads **small pieces**.

This is efficient for **large files**.

3️⃣ Create Write Stream

```
const writeStream = fs.createWriteStream("output.txt");
```

This creates a **Writable Stream**.

Meaning:

Node will write data into output.txt

So the program is preparing a file where the data will be written.

4️⃣ What is pipe()?

```
readStream.pipe(writeStream);
```

pipe() **connects two streams together**.

Flow:

readStream → writeStream

Meaning:

Read data from input.txt

↓

Send it directly

↓

Write it into output.txt

So the data **moves automatically from one stream to another**.

---

5. Full Flow of the Program

input.txt

↓

createReadStream

↓

pipe()

↓

createWriteStream

↓

output.txt

Result:

input.txt content → copied to → output.txt

---

6. Example

If **input.txt** contains:

Hello Node.js

Streams are powerful

After running the program:

node stream.js

The **output.txt** will contain:

Hello Node.js

Streams are powerful

So this code **copies a file using streams**.

7⃣ Why pipe() is Important

Without pipe() you must manually handle chunks.

Example (longer code):

```
readStream.on("data", (chunk) => {
  writeStream.write(chunk);
});
```

But pipe() does this **automatically and efficiently**.

🧠 Interview Answer

❓ What is pipe() in Node.js?

Answer:

pipe() is a method used to **transfer data from a readable stream to a writable stream automatically**.

📊 Your Course Progress

50% Complete

Next lesson we will learn **Buffers in Node.js**, which explains **how Node handles binary data internally**.

Just type **"next lesson"** 🚀

Great 👍 Let's continue.

You are now in **Lesson 10 of Node.js**.

Today we will learn **Buffers**, which are used by Node.js to handle **binary data** (like images, videos, files).

🚀 **Lesson 10 – Buffers in Node.js**

1 **What is a Buffer?**

A **Buffer** is a temporary memory area used to store **binary data**.

In **Node.js**, buffers are used when working with:

- files
- network data
- streams

Example:

File → Buffer → Process → Output

Why Buffers Are Needed

Normally JavaScript works with **strings and objects**.

But when reading files or network data, the data comes as **binary data**.

Example binary:

01010100 01100101 01110011 01110100

So Node stores this data inside a **Buffer**.

---

3 Create a Buffer

Example:

```
const buffer = Buffer.from("Hello");

console.log(buffer);
```

Output:

<Buffer 48 65 6c 6c 6f>

Explanation:

48 → H

65 → e

6c → l

6c → l

6f → o

---

4 Convert Buffer to String

Example:

```
const buffer = Buffer.from("Hello");


console.log(buffer.toString());
```

Output:

Hello

---

5 Create Empty Buffer

Example:

```
const buffer = Buffer.alloc(10);

console.log(buffer);
```

Output:

<Buffer 00 00 00 00 00 00 00 00 00 00>

This creates a **buffer with 10 bytes**.

---

6 Buffer Example

```
const buffer = Buffer.from("NodeJS");


console.log(buffer);
console.log(buffer.toString());
```

Output:

```
<Buffer 4e 6f 64 65 4a 53>
NodeJS
```

---

7 Where Buffers Are Used

Buffers are used internally in **Node.js** for:

- **Streams**
- **File system operations**
- **HTTP requests**
- **TCP connections**

Example flow:

File → Buffer → Stream → Output

□ Interview Question

❓ What is a Buffer in Node.js?

Answer:

A **Buffer** is a temporary memory area used to **store binary data while it is being transferred or processed in Node.js**.

📝 Practice Task

Create file:

buffer.js

Code:

```
const buffer = Buffer.from("Kartik");
```

```
console.log(buffer);

console.log(buffer.toString());
```

Run:

```
node buffer.js
```

📊 **Course Progress**

55% Complete

Completed topics:

✓ Node introduction
✓ Event loop
✓ Core modules
✓ NPM
✓ HTTP server
✓ Routing
✓ JSON APIs
✓ EventEmitter
✓ Streams
✓ Buffers

⏭ **Next Lesson (Very Important)**

Next we will learn **Process Object in Node.js**.

You will learn:

- process.argv
- environment variables
- controlling Node applications

These are used in **real backend apps**.

Type **"next lesson"** and we continue 🚀

Great 👍 Let's continue.

You are now in **Lesson 11 of Node.js**.

Today we will learn the **Process Object**, which lets your program interact with the **system running Node.js**.

🚀 Lesson 11 – Process Object in Node.js

1⃣ What is process?

process is a **global object** available in **Node.js**.

It gives information about:

- the current Node.js process
- system environment
- command-line arguments

Example:

console.log(process);

This prints a large object with system details.

2⃣ process.argv (Command Line Arguments)

argv means **argument vector**.

It stores arguments passed when running the program.

Example command:

node app.js hello world

Code:

console.log(process.argv);

Output:

```
[
 'node',
 'app.js',
 'hello',
 'world'
```

]

Explanation:

Index	Value
0	node path
1	file name
2	first argument
3	second argument

3⃣ Example Program

Create file:

args.js

Code:

```
const name = process.argv[2];


console.log("Hello " + name);
```

Run:

```
node args.js Kartik
```

Output:

```
Hello Kartik
```

4⃣ Environment Variables

Environment variables store **configuration values**.

Example:

```
console.log(process.env);
```

You will see many system variables.

Example variable:

PATH

HOME

USER

These help configure applications.

5⃣ Exit a Program

You can stop a program manually.

Example:

```
process.exit();
```

Example:

```
console.log("Start");
process.exit();
console.log("End");
```

Output:

```
Start
```

The program stops before "End".

6⃣ Process ID

Every running program has a **process ID**.

Example:

```
console.log(process.pid);
```

Output example:

```
12345
```

🧠 Interview Question

❓ What is the process object in Node.js?

Answer:

The **process object** in **Node.js** provides information about the current Node.js program and allows interaction with the system environment.

📝 **Practice Task**

Create program:

node greet.js Kartik

Output should be:

Welcome Kartik

📊 **Course Progress**

60% Complete

Completed topics:

✓ Node introduction
✓ Event loop
✓ Core modules
✓ NPM
✓ HTTP server
✓ Routing
✓ JSON APIs
✓ EventEmitter
✓ Streams
✓ Buffers
✓ Process object

⏭ **Next Lesson (Very Important for Backend)**

Next we will learn **Middleware** and start working with **Express.js**.

This is where **real backend development starts**.

Just type **"next lesson"** and we move to **Express.js** 🚀

No problem 👍 I'll explain **Lesson 11 (Process Object)** again **very simply**.

We are still working in **Node.js**.

---

1 What is process?

process is a **global object** in **Node.js**.

It gives information about:

- the running Node program
- system environment
- command line input

Example:

console.log(process);

This prints information about the **current Node program running in your computer**.

---

2 process.argv (Command Line Arguments)

argv means **arguments passed when running the program**.

Example command:

node app.js Kartik

Example code:

console.log(process.argv);

Output example:

```
[
 'C:\\Program Files\\nodejs\\node.exe',
 'C:\\Users\\app.js',
 'Kartik'
]
```

Explanation:

Index	Meaning
0	Node path
1	File name
2	First argument

---

3 Practical Example

Create file:

greet.js

Code:

```
const name = process.argv[2];

console.log("Hello " + name);
```

Run in terminal:

```
node greet.js Kartik
```

Output:

```
Hello Kartik
```

Explanation:

process.argv[2] → Kartik

---

4 process.env (Environment Variables)

Environment variables store **system settings**.

Example code:

```
console.log(process.env);
```

Output shows many system variables like:

PATH

HOME

TEMP

Developers use these to store things like:

database password

API keys

port numbers

---

5 process.exit()

This **stops the Node program immediately**.

Example:

```
console.log("Start");

process.exit();

console.log("End");
```

Output:

```
Start
```

The program stops before "End".

---

6 process.pid

Every program running on a computer has a **process ID**.

Example:

```
console.log(process.pid);
```

Output example:

```
4567
```

This number identifies your running Node program.

---

7 Real Example

Example program:

```
console.log("Program name:", process.argv[1]);

console.log("User input:", process.argv[2]);

console.log("Process ID:", process.pid);
```

Run:

```
node test.js hello
```

Output:

```
Program name: test.js

User input: hello

Process ID: 3456
```

□ Simple Way to Remember

process → information about running Node program

Important parts:

process.argv → command line input

process.env → system variables

process.exit() → stop program

process.pid → process id

📊 **Your Course Progress**

60% Complete

Next we will start **backend framework**:

Express.js

This is where you will start building **real APIs for your MERN projects**.

Just type **“next lesson”** and we start **Express.js** 🚀